

Sharif University of Technology

Department of Mathematical Sciences

A Survey on Cryptographic Voting Systems

Submitted by:
Nadia Afsar

Submission Date: September 26, 2026

Abstract

This report surveys different techniques for implementing cryptographic voting systems. It explores approaches to building mix-nets for anonymizing votes in election systems, and examines mix-net-based systems such as the Chaumian mix-net. We then examine Scratch & Vote, which does not require any mix-net, and Helios, the first web-based open-audit voting system. We observe that each approach has its own advantages and disadvantages, and that a trade-off must be made based on the specific requirements of the election. Finally, we examine return code schemes, which are used to verify that a vote is cast as intended.

Table of Contents

Introduction	1
Preliminaries and Definitions	1
Challenges.....	3
Mix-nets	4
Randomized Partial Checking	5
A Mix-Net From Any CCA2 Secure Cryptosystem.....	6
Scratch & Vote.....	8
Helios	9
Return Code Schemes	11
Conclusion	12
References	13

Introduction

Voting has always been a critical concern throughout history. In democratic societies, elections are needed for any decision that is about to be made, and their most important role has been choosing the people who will rule their communities. Since these processes are highly sensitive, it is crucial that they are conducted correctly and reliably. However, ensuring both correctness and secrecy in elections has proven to be a significant challenge, one that cryptography aims to solve.

Various voting methods have been employed over time, and many remain in use today. However, most of these systems lack proper security considerations and are vulnerable to cheating or manipulation. Traditional systems rely on trusted authorities and provide no verifiable proof of the election's correctness, which is not practical for ensuring election integrity.

Due to the importance of election security, cryptography researchers have been trying to design voting protocols using mathematical tools. Their goal is to ensure that elections are conducted correctly while keeping voters' choices hidden, so that coercers cannot influence the outcome.

This report provides a survey on cryptographic contributions to secure voting. First, we will explore classic voting systems and the challenges that limit their practicality. Then, we will introduce cryptographic voting systems, their properties, and the improvements they have made to election processes. We will also discuss several techniques for improving these systems.

These systems have improved over the years but still require further development. Some of them are vulnerable to several attacks and are risky to deploy; nevertheless, studying them is valuable, since they may be suitable for special scenarios or their ideas may inspire more robust systems.

Preliminaries and Definitions

Homomorphic Encryption

Homomorphic encryption is a type of encryption that allows computations to be performed on ciphertexts without decrypting them. More precisely, it is a cryptosystem where an operation on two ciphertexts corresponds to the encryption of a specific (possibly different) operation on their underlying plaintexts.

Homomorphic encryption enables tallying to be performed directly on encrypted votes. The individual votes are never decrypted; instead, the aggregation is computed homomorphically, and only the final tally is decrypted. In this setting, there is no need for a separate anonymization phase such as a mix-net. However, mix-net-based systems also rely on homomorphic properties for re-encryption: each mix server must be able to re-encrypt ciphertexts without changing their plaintext, which is precisely what a

homomorphic (rerandomizable) cryptosystem allows.

For example, in the El Gamal cryptosystem, for two plaintexts m_1 and m_2 , we have:

$$\begin{aligned}\text{Enc}(m_1) \otimes \text{Enc}(m_2) &= (g^{r_1}, m_1 \cdot y^{r_1}) \otimes (g^{r_2}, m_2 \cdot y^{r_2}) \\ &= (g^{r_1+r_2}, (m_1 \cdot m_2) \cdot y^{r_1+r_2}) = \text{Enc}(m_1 \cdot m_2).\end{aligned}$$

Thus, El Gamal is multiplicatively homomorphic.

Zero-Knowledge Proof

Zero-knowledge proofs (ZKPs) allow a prover to convince a verifier that a statement is true without revealing anything beyond the truth of the statement. These proofs are usually interactive; however, if the proof requires no interaction, it is called a non-interactive zero-knowledge proof (NIZKP). In voting systems, we primarily require NIZK proofs.

Secret Sharing

Vote encryption is typically performed using public-key cryptography, and decryption requires a shared secret key distributed among officials. This ensures that no single person can decrypt the votes; instead, a group of officials must cooperate to reveal the result. This technique, known as secret sharing, enables secure multi-party computation and makes large-scale collusion unlikely. As a result, such systems provide a practical balance between security and trust.

Secret sharing is a cryptographic technique where a secret s is divided into n shares and distributed among n parties. The scheme is designed such that any subset of at least k shares can reconstruct the secret, while any subset of fewer than k shares reveals no information about it. In the context of voting, this ensures that no single official can decrypt votes; instead, a threshold of k officials must cooperate to perform decryption.

One approach for this purpose is the one introduced by Shamir. In this implementation, the secret s is chosen as $P(0)$, where P is a polynomial of degree $k - 1$. Based on Lagrange interpolation, k distinct points $(x_i, P(x_i))$ are required to uniquely determine P . Therefore, each share is designed to give a different point on the polynomial, ensuring that k shares suffice to recover s , while fewer than k reveal nothing.

Another well-known secret sharing scheme is the one proposed by Asmuth and Bloom, which relies on the Chinese Remainder Theorem (CRT). In this scheme, a special modulus m_0 is chosen such that $m_0 > s$, where s is the secret. Then, a set of pairwise coprime moduli $m_1, m_2, \dots, m_n$ are selected. To share the secret, a random number α is generated, and the value $x = s + \alpha \cdot m_0$ is computed. The shares are then defined as the remainders $x_i \equiv x \pmod{m_i}$ for each party. Because the moduli are pairwise coprime, the CRT guarantees that combining at least k shares uniquely determines x modulo the product of those k moduli. Finally, the original secret is recovered by computing $s = x \bmod m_0$.

CCA2 Security

CCA2 security is a strong notion of security for public-key encryption: even an adversary with access to a decryption oracle (except for the challenge ciphertext) cannot distinguish which of two messages is encrypted in the challenge ciphertext. This property is required in the CCA2-based mix-net to prevent an adversary from creating related ciphertexts that could break privacy during tracing.

Message Authentication Code

A message authentication code (MAC) is a cryptographic primitive that allows two parties sharing a secret key to verify the integrity and authenticity of a message. It is secure if it is existentially unforgeable under chosen-message attacks, meaning no efficient adversary can forge a valid tag for a new message even with access to a tagging oracle.

Challenges

Voting began with simple methods, such as raising thumbs to show agreement. Over time, paper ballots were introduced, followed by the use of isolated booths to protect voters' privacy. Lever machines later increased the speed and efficiency of voting, and punch cards allowed voters to select multiple options on a single card. Later, Direct Recording by Electronics (DRE) machines were introduced, making elections faster and more convenient. However, despite these technological advancements, none of these systems provided a cryptographic proof of correctness. They all relied on trust in officials and equipment, which is not enough for secure elections.

Secrecy

The first major issue is secrecy. Classic voting systems attempted to address it, but with limited success. A coercer may try to force a voter to vote for a specific candidate. In this situation, the voter would need to be able to prove to the coercer that they followed instructions—yet the system must prevent exactly this. Thus, a secure voting system must guarantee that no one can learn another person's vote, and that voters cannot produce evidence of their choice to any third party.

Verifiability

The second major challenge is verifiability. Classic voting systems rely entirely on trust in officials, but officials may have incentives to cheat. Voters need assurance that their votes were recorded correctly, and the public needs to be confident that the tallying was conducted without errors. Therefore, a practical voting system must provide proof at every stage of the process.

For these reasons, cryptographic voting systems emerged. In such systems, votes are encrypted—typically using homomorphic or rerandomizable encryption—and the voter receives a receipt to verify the encryption. The encrypted vote is then posted on a public bulletin board, along with a Non-Interactive Zero-Knowledge Proof (NIZKP) that the ballot is well-formed and that the voter knows the necessary credentials. This allows anyone to verify the submission. Officials then anonymize the votes through a verifiable shuffle, providing a ZKP that the mixing was done correctly. Finally, the anonymized votes are decrypted and tallied to produce the final result.

Mix-nets

Systems that require anonymization after vote encryption typically rely on *mix-nets*. A mix-net consists of a sequence of mix servers. Each server takes the encrypted votes, shuffles their order, re-encrypts them, and forwards them to the next server. To ensure correctness, each server must produce a zero-knowledge proof that no votes were removed or altered. As long as at least one server in the chain is honest, the anonymization is guaranteed.

The privacy of a mix-net relies on computational assumptions: a computationally unbounded adversary could simply decrypt the inputs and outputs to discover the permutation. Beyond this, the level of privacy also depends on whether the proofs leak information about the permutation. In the strongest case, called complete and independent privacy, all permutations remain possible, and revealing some input-output links does not leak information about others. Weaker notions, such as complete but dependent or incomplete privacy, arise when the proofs themselves narrow the set of possible permutations.

Unlike privacy, the soundness of mix-nets does not necessarily rely on hardness assumptions. What matters is the guarantee provided by the soundness proofs. If even a computationally-unbounded prover has only a negligible chance of cheating, the proof is called overwhelming. If the proof only ensures that a computationally-bounded prover has a negligible chance of cheating, it is called an overwhelming argument. Finally, if the chance of cheating is small but not negligible, the system is said to offer high but not overwhelming soundness (either as a proof or an argument, depending on the prover's computational bounds).

Chaumian Mix-net

The first mix-net was introduced by Chaum, using RSA for encryption. The idea was that each voter pads their vote with a random string and encrypts the result with a public key belonging to a specific mix server. They then pad it with another random string and encrypt it with another public key. This process is repeated several times. During decryption, each mix server decrypts its layer in order, removes the padding, and passes the result to the next server. To ensure anonymization, the votes are shuffled at each decryption round.

The verification mechanism is particularly clever. First, the voter chooses a pair of public and secret keys. They encrypt their public key using the layered encryption described above and send it through the mix-net. Later, they check the bulletin board to ensure their public key appears, confirming that the mix servers are operating honestly.

In the real voting round, the voter encrypts their vote with their own secret key and attaches their public key to it. This entire package is then encrypted using the same multi-layer process. This approach ensures that only the person who possesses the voter's secret key can cast a vote on their behalf, while anyone can later decrypt the vote using the attached public key. In other words, each voter verifies the mixing process, and then everyone can verify that the tallying was carried out without cheating.

The fundamental weakness of this construction is the multiplicative homomorphism of raw RSA. An attacker can exploit this by multiplying a target ciphertext by the encryption of a small constant f . When

the mixnet decrypts the outputs, this constant remains as a factor in the plaintext. Since no other output will have the same algebraic relationship to a known input, the attacker can uniquely identify the output corresponding to the targeted vote, effectively tracing it through the mixnet.

Re-encryption Mix-nets

The large size of encrypted votes, together with privacy concerns, motivated the development of re-encryption mix-nets. In these schemes, rerandomization is done algebraically: the new randomness is combined with the existing randomness rather than appended. Cryptosystems like El Gamal support this property, as they allow a server to add fresh randomness without knowing the original random value. Each mix server re-encrypts the ciphertexts, shuffles them, and passes them to the next server. Only after all shuffles are complete is the final decryption performed.

Sako–Kilian Mix-net

One example of a re-encryption mix-net is the Sako–Kilian mixnet. In this construction, each mix server receives the ciphertexts, re-encrypts them with fresh randomness, permutes them according to a random permutation, and passes them to the next server. The server provides a proof that the shuffling was done correctly as follows: it prepares a second, “shadow” mix and receives challenges from the verifier. If the challenge is 0, the server reveals the shadow mix; if the challenge is 1, the server reveals the difference between the main mix and the shadow mix. It is clear that if the server cheats, its chance of not being detected is $1/2$. This soundness error can be reduced by repetition. Since this proof is public-coin, it can be transformed into a non-interactive zero-knowledge proof via the Fiat–Shamir transformation.

Although this approach improves privacy, it does not eliminate all vulnerabilities. The algebraic structure of the ciphertexts still reveals a relationship between inputs and outputs, which an attacker can exploit to infer input-output mappings.

Another major drawback of these early mix-nets was their reliance on trust in the mix servers. This limitation led to the development of Universally Verifiable mix-nets, where each server provides a zero-knowledge proof that it has not removed or altered any vote, and that the shuffling corresponds to a valid permutation.

Subsequent developments focused on improving the efficiency of these proofs while maintaining the required privacy guarantees.

Randomized Partial Checking

RPC is a technique for proving the correctness of mix-nets. The idea is to reveal how a randomly selected subset of ciphertexts was processed. An advantage of RPC is that it is compatible with any cryptosystem for which a mix server can efficiently prove that it processed a single ciphertext correctly. This makes RPC suitable for electronic voting.

Construction

Here is a general construction for RPC. Before the mixing process begins, each mix server M_j picks a random permutation π_j . The servers must then commit to their permutations so that they cannot alter them during the process. M_j commits to either π_j or π_j^{-1} , depending on whether j is odd or even. Then the mixing starts. Each server receives the ciphertexts, permutes them according to its committed permutation, and applies a cryptographic transformation to the newly ordered ciphertexts. This transformation depends on the type of mixnet: in a Chaumian mixnet, the server decrypts each vote with its share of the private key; in a re-encryption mixnet, the server re-encrypts the votes. For verification, the mix servers jointly select a random subset of each server's inputs or outputs (depending on whether the server is even or odd), and the server is asked to reveal the corresponding input, the resulting output after permutation and transformation, and the randomness used for the transformation. The server also reveals the commitment for each selected correspondence. If all revealed pairs verify correctly, the server passes the correctness check; otherwise, it is identified as a cheater.

Improvements

If the mixnet does not remove duplicate votes, it allows an adversary who has corrupted some senders and the first and last mix servers to attack the privacy of votes. The idea is that the adversary targets specific votes and encrypts each of them a certain number of times. As a result, the adversary can identify the decryption of each targeted vote using the number of times it was repeated. Thus, by removing duplicate votes, such an attack becomes impossible.

Another problem that should be considered is the inconsistency of the committed permutation. If an adversary corrupts a mix server and commits to the same index twice, it can then replace one of the ciphertexts whose committed index points to that output with a malicious ciphertext (a re-encryption of a product of the targeted votes). Because both commitments open to the same output index, the server can always reveal the other, valid correspondence when challenged, and the malicious ciphertext remains hidden. As a result, the adversary can learn the decryption of the targeted votes using the algebraic relation between the targeted ciphertexts. If consistency is not checked, this attack can be performed without any possibility of detection. Another attack that may be possible when consistency is not checked is that an adversary can commit all permutation commitments to the same index, then replace every ciphertext with a re-encryption of a single chosen vote. In this way, the adversary can change the result of the election.

The attack mentioned for the case where inconsistency is not checked can still be performed even if the consistency of the revealed commitments is verified. This attack succeeds if the altered ciphertext is not selected during verification, and it remains undetected in that case.

Therefore, RPC's security is not rigorously established, and deploying it may be risky and requires careful consideration.

A Mix-Net From Any CCA2 Secure Cryptosystem

Overview and Motivation

As mentioned earlier, previous approaches to mix-nets had limitations: they either required homomorphic cryptosystems or lacked efficient proofs. Now we introduce a mix-net that can be constructed using any CCA2-secure cryptosystem. This mix-net is based on Chaum's mix-net with several modifications. We will first discuss the overall process and then explain the reason for each modification.

Key Setup and Voter Encryption

This mix-net requires five key pairs: one for the outer encryption layer, one for the repetition encryption layer, one for the final encryption layer, and two for the two Chaum's mix-nets. First, each voter encrypts their vote with the final layer public key and makes t identical copies of it, then encrypts them with the repetition layer public key. Next, they encrypt the ciphertexts with the public keys of the Chaum's mix-net with partial tracing and then the mix-net with explicit verification. Finally, they concatenate them and encrypt the resulting ciphertext with the outer layer public key and submit it for further processing.

Mixing and Verification Process

Before initiating the decryption process, each mix server adds an encrypted dummy value to the encrypted votes. First, the ciphertexts are decrypted with the secret-shared key of the outer layer. The result consists of concatenated bitstrings, each of which is then split into t individual ciphertexts using the Split_t function. Then the mix-net with explicit verification is run as a normal Chaum mix-net, except it does not provide any proof during decryption, and the secret keys are revealed publicly after the second mix-net to prove it was executed correctly. After that, the mix-net with partial tracing is run. Then the ciphertexts are decrypted with the secret-shared key of the repetition layer, and the ciphertexts that do not have exactly t copies are identified. The mix-net traces these ciphertexts backward to identify the origin of the broken copysets (either a corrupted sender or a cheating mix-server). The mix-net also traces the dummy values forward, and each server reveals its action to prove it decrypted honestly. Finally, duplicates are removed using the Unique function, the remaining ciphertexts are sorted, and the ciphertexts are decrypted with the secret-shared key of the final layer.

Design Rationale and Limitations

The final encryption layer preserves the privacy of honest voters' choices during the tracing steps. Encrypting t copies of each vote prevents malicious servers from cheating, since they cannot identify which ciphertexts encrypt the same vote. The repetition encryption layer prevents the last server of the second mix-net from having access to the decrypted ciphertexts earlier than everyone else and thus from having the chance to cheat. The mix-net with partial tracing plays the role of the main mix-net. To prevent the first mix-server from knowing which ciphertexts belong to the same copyset, a mix-net with explicit verification is added. Dummy values help prevent servers from cheating, since to replace all ciphertexts they would have to replace the dummy values as well. The outer layer prevents partial copying of an honest sender's bundle: by CCA2 security, an adversary can either copy the entire bundle (which is then detected as a duplicate) or create an independent bundle (whose collision with the honest bundle is negligible).

This construction is a significant improvement over Chaum's mix-net because it requires minimal cryptographic assumptions and only needs a CCA2-secure cryptosystem. This is particularly interesting because homomorphic cryptosystems are not known to be secure against post-quantum computers. The security of this mix-net is provable, in contrast to RPC. However, it does not provide universal verifiability.

Scratch & Vote

Although significant improvements have been made to mixnets in terms of efficiency and privacy, they still require expensive zero-knowledge proofs to guarantee correct mixing. It is therefore desirable to avoid this cost altogether. Homomorphic encryption provides a way to do so: it allows tallying to be performed directly on encrypted votes, without ever decrypting them individually. This preserves voter secrecy while eliminating the need for complex shuffle proofs.

Scratch & Vote systems exploit this property of homomorphic encryption. In these systems, when a vote is submitted, it is encrypted using a homomorphic scheme and posted on a public bulletin board, accompanied by a non-interactive zero-knowledge proof that the vote is well-formed. The voter also receives a receipt, which allows them to later verify that their vote has been correctly recorded. The tallying process is then performed directly on the encrypted votes, without decrypting them individually. Finally, only the aggregated result is decrypted.

Here are the details: A key pair is generated by the election officials, and each official holds a share of the secret key. Voting takes place in an isolated booth. The scheme is designed for pre-determined candidates: the candidate names are printed in a random order on the left half of the ballot, while the voter marks their choice on the right half. The right half also contains a scratch surface, which hides the randomization values used to encrypt each candidate choice, in the same order as the candidate names. A 2D barcode on the ballot encodes the ciphertexts corresponding to each candidate; scanning the barcode retrieves these ciphertexts. Before leaving the booth, the voter detaches and discards the left half of the ballot, so that no one can see the candidate order. After the voter submits the right half, an election official first verifies that the scratch surface is intact. The ballot is then scanned, and the ciphertext corresponding to the chosen candidate is posted on the public bulletin board, along with a non-interactive zero-knowledge proof that the ballot is well-formed. Finally, the scratch surface is detached and discarded, and the voter takes the remaining portion of the ballot home as a receipt.

Verification

The voter verifies that the encryption is performed correctly using a cut-and-choose approach. They take an additional ballot and remove its scratch surface. They then scan the barcode and check whether the generated ciphertexts correctly encrypt the candidate choices. Since the scratch surface has been removed, this ballot is void and cannot be cast. Everyone can verify that only valid ballots are cast, using the provided NIZK proofs. Because tallying is performed homomorphically on the encrypted votes, anyone can reproduce the tally and verify that the officials carried it out correctly. Decryption requires the cooperation of a threshold of officials, thanks to secret sharing, and is accompanied by zero-knowledge proofs of correct decryption. Thus, an attempt to cheat during decryption is unlikely to succeed or go undetected.

Privacy

Vote casting takes place in an isolated booth, so no one can observe the voter's choice. Moreover, the left half of the ballot is discarded by the voter, and the scratch surface is removed and discarded by an election official. As a result, the voter cannot prove to others how they voted, and it is unlikely that a coercer could influence the voter's choice. This separation is what provides incoercibility: the voter has enough information to verify their own vote, but not enough to convince anyone else

Improvement

If an election includes multiple races, the Prêt-à-Voter-based design results in a cluttered and impractical ballot. One improvement is to change the ballot design entirely. The adapted system uses two separate sheets. The top sheet contains the candidate names in a standard order, along with the scratch surface and the barcode. The bottom sheet contains only the candidate codes in a random order. The voter marks their choices on the top sheet, and then detaches and discards the bottom sheet before leaving the booth. Using two separate sheets allows multiple races to be accommodated more neatly, while preserving the security properties of Scratch Vote. The clever aspect of this adaptation is that it preserves Punchscan's multi-race advantage while eliminating its main weakness: the voter's freedom to choose which sheet to keep, which could otherwise enable vote selling.

Drawbacks

Scratch & Vote appears to offer a good trade-off between verifiability and privacy. However, it has several limitations. It does not support write-in votes, since candidates are pre-determined and voters cannot choose anyone outside the predefined list. In addition, generating a NIZK proof for each ballot is expensive.

The system is also vulnerable to several attacks. The randomness of the candidate ordering depends on election officials, who may cheat by providing non-random orderings or malformed ballots. They may also record the candidate orderings and thereby compromise ballot secrecy. This can be mitigated if ballot generation is automated and the machines do not retain the randomness used.

One attack from the voter's side is chain voting: the voter may succeed in casting a pre-marked ballot previously received from a coercer. Another interesting attack is coerced randomization: the coercer asks the voter to vote for the first candidate on the list, and this can be checked from the public bulletin board. This cannot force the voter to vote for a specific candidate, but it prevents them from voting for a particular candidate and reduces that candidate's chance of winning.

Helios

Ben Adida designed Helios, the first web-based open-audit voting system, primarily intended for online communities, local clubs, and student government elections. Since the motivation for coercion is lower than in political elections, and such elections are not as sensitive, Helios focuses mainly on unconditional integrity rather than privacy. Helios is designed with the potential to support write-in votes in future

versions, which is why it uses a mixnet: tallying without decryption is not possible when write-in votes are allowed. Helios allows any auditor to verify that votes are encrypted correctly. The encrypted votes are then shuffled using a mixnet, and NIZK proofs are provided for the shuffle. Finally, the shuffled ciphertexts are decrypted and tallied, with NIZK proofs also provided for the decryption step.

Now let us look at the details. Anyone can create an election and is considered the administrator. The administrator enters the information needed for the ballot and can also view the list of voters and remove a voter if desired. After the election is frozen, no changes to the list are possible and voting begins.

Voting and Auditing

When a voter selects an option, their choice is encrypted with El Gamal and hashed with SHA-1. The hash of the ciphertext is shown to the voter, and they can choose to audit the encryption. In that case, the randomness used for encryption is revealed, and the voter can verify that the encryption was performed correctly. If they decide to audit, their vote is then encrypted again with fresh randomness. They can audit as many times as they wish until satisfied. If they choose to cast the vote, they are asked to authenticate their identity to ensure they are eligible to vote. Because authentication happens only at this step, people who are not eligible to vote can still audit the encryption. The voter also has the “Coerce Me!” option, which provides a proof of how they voted that they can share with anyone; the vote is not re-encrypted. Note that this proof does not include any detail about whether the vote was actually cast, so this option does not weaken privacy.

Mixing, Decryption, and Tallying

Afterward, the encrypted votes are posted on the bulletin board. Helios uses the Sako–Kilian approach for mixing them, with only one server. It uses the non-interactive version of the shadow-mix proof to convince the auditors. Then it decrypts the votes and provides proofs for them using Chaum–Pedersen, which is an honest-verifier zero-knowledge proof and can be transformed into a non-interactive proof via the Fiat–Shamir transformation. Finally, the tallying is performed on the decrypted votes.

Verification

The encryption of a ballot can be verified by anyone using the audit option mentioned earlier. In addition, proofs of shuffling and decryption are provided as NIZK proofs, and any auditor can access them. Helios provides an Election Tallying Verification program that downloads the list of cast votes, the shuffled votes, and the proofs of shuffling and decryption. After verifying the proofs, it re-performs the tally to ensure it was performed correctly. The key point is that anyone can run this program, so if the system removes or alters any vote, an auditor who independently publishes the complete list would detect the discrepancy.

Privacy

Although we noted that this system prioritizes integrity over privacy, some measures are still taken to protect privacy. The voting interface is designed so that no data is transmitted over the network during ballot preparation, before encryption. When the voter opens the web page, all necessary data is loaded; the voter can then disable the network connection to ensure that no information leaks.

The interesting point about Helios is that it does not require trust in the server for integrity. Instead, it allows any auditor to verify the entire process. However, the system can be improved to provide better privacy. In particular, it can rely on multiple servers for shuffling instead of one. This would improve privacy because, as long as at least one of the servers is honest, the link between voters and their ballots remains hidden. Currently, privacy depends on trusting the single Helios server; with multiple trustees, privacy would only require that a single one of them is honest, which is a much weaker assumption.

Return Code Schemes

Return code schemes are an approach to verifying that a vote is cast as the voter intended. The idea of return codes is that the voter sends their encrypted vote to the election server and receives a message authentication code (MAC) tag that verifies the vote was cast as intended. We can construct such schemes with homomorphic encryption systems, because using them the servers can generate a MAC related to the chosen vote without decrypting the received ciphertext.

A first idea for the construction is as follows. Each voter receives a code sheet through a trusted channel in the setup phase, listing the return code $\text{code}_j = m_j^{a_i} \beta_i$ for every pre-defined option m_j . The i -th voter encrypts their chosen vote and sends it to the servers along with a ZK proof that it is an encryption of one of the pre-defined choices. The servers then raise the ciphertext to the random element a_i , which is chosen for voter i in the setup phase and secret-shared among the servers, and multiply the result by an encryption of the random element β_i , used as a one-time pad (OTP). By the homomorphic property of the encryption scheme, the resulting ciphertext encrypts $m_j^{a_i} \beta_i$, which is the MAC tag. The servers jointly decrypt it and return the MAC tag to the voter. Because a_i is secret-shared and never reconstructed, the servers cannot recover the vote from the MAC tag. After receiving the MAC tag, the voter verifies that it matches the code printed next to their chosen option on the code sheet. This scheme supports write-in votes since it is based on an unconditionally secure one-time MAC, which remains secure for any message, not just the pre-defined choices. Since a pre-printed code sheet cannot cover all possible write-in values, the voter needs the shared MAC key to verify the return code for an arbitrary message.

We can choose a random representative $\beta_{i,j}$ for each choice of each voter to eliminate the need for a per-voter secret-shared exponent. The return code is then $\beta_{i,j} \beta_i$, where β_i is known to the servers only in encrypted form for each voter. Such an approach allows re-voting, in contrast to the previous construction. This scheme does not fully support write-in votes because its security relies on the message being a random representative unknown to the adversary; for a write-in, the voter chooses the message, so it is no longer random, and if the adversary can guess it, they can forge return codes for other messages.

We can even eliminate the need for $\beta_{i,j}$ by using hash functions. To do this, a standard MAC scheme is combined with the one-time pad, where each server holds its own secret key. The construction works as follows: $E_y(\beta_i)E_y(m) = E_y(\beta_i m)$ is computed, where $E_y(m)$ is received from the voter and $E_y(\beta_i)$ is as defined before; the result is then decrypted. Each server computes a MAC on $\beta_i m$ using its own key, and the resulting MACs are hashed together. This approach no longer requires a separate secret-shared element for each voter. However, the drawback is that the return code cannot be generated in the absence of a server. This scheme does not support write-in votes because the return code is computed by the servers using their own secret MAC keys, which the voter does not know. For a write-in, the voter would need to verify the return code for an arbitrary message, but this is impossible without access to the MAC keys, and a pre-printed code sheet cannot cover all possible write-in values.

The return code schemes discussed so far require the servers to be online during the voting phase. The idea for resolving this problem is to generate a pair of ciphertexts $(v_{i,j}, w_{i,j})$ for each choice j of each voter i , so that the first element is submitted as the vote and later tallied offline, while the second element is used for generating the return code online. This is achieved by using two independent public keys: y for the actual vote and z for the return code. The online servers only need the secret key for z , so they can generate the return code without touching the vote. The tallying servers, which hold the secret key for y , can then process the votes offline, behind an airwall. This way, cast-as-intended verification is preserved while tallying no longer requires online servers.

Conclusion

We began this survey by discussing why classic voting systems are no longer sufficient and why cryptographic voting systems are needed. Then we examined the role of mix-nets in the construction of election systems, their weaknesses, and how they can be improved. Chaum's mix-net was an interesting starting point, but it allowed many attacks, some of which were mentioned. This led to the construction of re-encryption mix-nets, which resolved some problems but were still not sufficiently strong. Then we introduced the RPC technique, which was proposed to verify the correctness of mix-nets. We discussed some of its flaws and possible fixes, but in the end it proved unreliable and required caution in its use. After that, we examined an approach to constructing mix-nets from any CCA2-secure cryptosystem, which appears more promising compared to previous constructions.

Next, we introduced Scratch & Vote, which does not require any mix-net. This system relies on homomorphic cryptosystems, which is an important disadvantage in a post-quantum setting. However, omitting the need for mix-nets makes it very efficient, since it avoids zero-knowledge proofs for each mix server.

Then we discussed Helios, a web-based voting system that does not promise strong coercion resistance, but is a helpful tool for elections among communities whose members lack pre-existing social ties. Its main advantage is its unconditional integrity.

Finally, we introduced return code schemes, which are used to verify that votes are cast as intended. These schemes use MAC tags for this purpose and are mainly based on homomorphic cryptosystems.

As we have seen, many improvements have been made in each approach to constructing election systems, but each fix introduces new challenges, and no single construction has been found that solves all problems simultaneously. It seems that the best that can be done is to strike a balance among the desired properties based on the purpose for which the voting system is used.

References

- [1] B. Adida, “Advances in Cryptographic Voting Systems,” Ph.D. dissertation, Dept. of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, Cambridge, MA, USA, 2006. [Online]. Available: <http://assets.adida.net/research/phd-thesis.pdf>
- [2] B. Adida, “Helios: Web-based Open-Audit Voting,” in *Proceedings of the 17th USENIX Security Symposium*, San Jose, CA, USA, Jul. 2008, pp. 335–348.
- [3] S. Khazaei, T. Moran, and D. Wikström, “A Mix-Net from Any CCA2 Secure Cryptosystem,” in *Advances in Cryptology – ASIACRYPT 2012*, Beijing, China, Dec. 2012, pp. 607–625.
- [4] S. Khazaei and D. Wikström, “Randomized Partial Checking Revisited,” Cryptology ePrint Archive, Report 2012/063, 2012. [Online]. Available: <https://eprint.iacr.org/2012/063>
- [5] S. Khazaei and D. Wikström, “Return Code Schemes for Electronic Voting Systems,” in *Electronic Voting – Second International Joint Conference (E-Vote-ID 2017)*, Bregenz, Austria, Oct. 2017, pp. 198–209.
- [6] I. F. Yildirim, “A Survey on Cryptographic Voting Schemes,” Sabancı University, Istanbul, Turkey, 2013. [Online]. Available: <https://research.sabanciuniv.edu/id/eprint/14131/1/IsmailFatihYildirim.pdf>